Citation Graph Builder for Human Evolution Literature

Mini Project for DS3294: Data Science Practice

Anusa Maiti Aritra Paul Aditi Nagrale Harshiddh Parmar

April 3, 2026

1 Introduction

Understanding how scientific knowledge evolves is crucial for researchers, funding agencies, and science historians. Citation graphs—where papers are nodes and citations are directed edges—provide a powerful framework to analyse influence, detect emerging trends, and map intellectual lineages. This project builds a fully automated pipeline that constructs a citation graph from academic literature in the field of human evolution. Starting from a set of seed papers (e.g., on Neanderthal genomics, hominin fossils, and archaic introgression), the system collects metadata from multiple APIs (Europe PMC, PubMed Central, arXiv, bioRxiv), extracts reference lists, resolves ambiguous citations using a cascade of matching rules, and stores the resulting directed graph in an SQLite database. A Streamlit dashboard allows interactive exploration of the graph, computation of centrality metrics, and visualisation of citation neighbourhoods. The goal is to provide a reproducible, scalable tool that can be adapted to any research domain.

2 Related Work

Citation analysis has a long history, from Garfield's impact factor to modern network science. Tools such as CiteSpace, VOSviewer, and Publish or Perish offer visualisation and bibliometric indicators, but they often require manual data curation or proprietary databases. On the other hand, large-scale infrastructures like the Open Citations Corpus (OCC) and Semantic Scholar provide open citation data, but they may not cover specialised fields exhaustively. Our work differs by focusing on a specific subdomain (human evolution), integrating four distinct data sources, and implementing a hybrid matching strategy (DOI, PMID, exact title, fuzzy title+author+year) to maximise recall. Unlike many existing pipelines that rely on PDF parsing, we use API-based reference extraction (Europe PMC, Semantic Scholar), which is faster and more reliable. The resulting graph is stored in a lightweight SQLite database, making it easy to query and share.

3 Our contribution

We contribute a complete, ready-to-run software package that:

- Collects paper metadata from four APIs with automatic rate-limiting and relevance filtering.
- Extracts citation lists via Europe PMC and Semantic Scholar without parsing PDFs.

- Resolves references to corpus papers using a multi-stage cascade (DOI → PMID → exact title → fuzzy matching).
- Builds a directed citation graph (nodes: papers, edges: citations) stored in SQLite, CSV, and Gephi-compatible formats.
- Provides an interactive Streamlit dashboard to query, visualise, and analyse the graph (degree distributions, centrality, ego networks).
- Includes a fallback PDF downloader (via PMC OA service and Unpaywall) and a debug tool for inspecting broken downloads.

All code is open source, documented, and designed to be extensible to other research fields.

4 Methods

4.1 Data

We collect papers published from 1990 onward that possess at least one structured identifier (DOI, PMID, or PMCID). Seed queries cover key topics in human evolution (e.g., "neanderthal genome ancient dna", "homo naledi new species", "archaic human introgression"). For each query, we poll Europe PMC, PubMed Central, arXiv (q-bio.PE category), and bioRxiv (evolutionary biology). Retrieved papers are deduplicated by DOI. Optionally, citation expansion (BFS depth 1) pulls in papers that cite or are cited by the seed set, subject to relevance filtering (inclusion/exclusion keyword lists).

4.2 Reference extraction and entity resolution

For every paper in the corpus, we fetch its reference list using the Europe PMC '/references' endpoint (fallback: Semantic Scholar). Each reference is then resolved to a node in the graph using the following cascade:

1. Exact DOI match.
2. Exact PMID/PMCID match.
3. Exact title match (case-insensitive, punctuation normalised).
4. Fuzzy match on title + first author + year (Jaccard similarity ≥ 0.55).

Unresolved references become placeholder nodes with the extracted metadata. The resolution method is recorded in the output edge list.

4.3 Graph construction and storage

We model the graph as $G = (V, E)$ where V is the set of papers and E is the set of directed citations ($p_i \to p_j$ means p_i cites p_j). The graph is stored in three complementary formats:

- `nodes_list.csv` and `edges_list.csv` for easy import into analysis tools.
- `citation_graph.db` (SQLite) with indexes on paper IDs and citation pairs.
- `network_map.gexf` for visualisation in Gephi.

NetworkX is used to compute basic statistics (in-/out-degree, weakly connected components, centrality), and the dashboard provides interactive subgraph visualisations.

5 Results

5.1 Graph statistics

Running the pipeline on a seed set of 57 papers with expansion depth 1 yields a graph with approximately 57 nodes and 230 directed edges. The largest weakly connected component contains 53% of the nodes. Papers with the highest in-degree are landmark reviews and methodological articles (e.g., on ancient DNA retrieval and hominin phylogenetics).

The Streamlit dashboard (Figure 1) allows users to:

- Search for a paper by title, author, or DOI.
- Display its ego network with an interactive force-directed layout.
- Compute and sort papers by degree centrality, betweenness centrality, or PageRank.
- Export the current subgraph as a PNG or CSV.

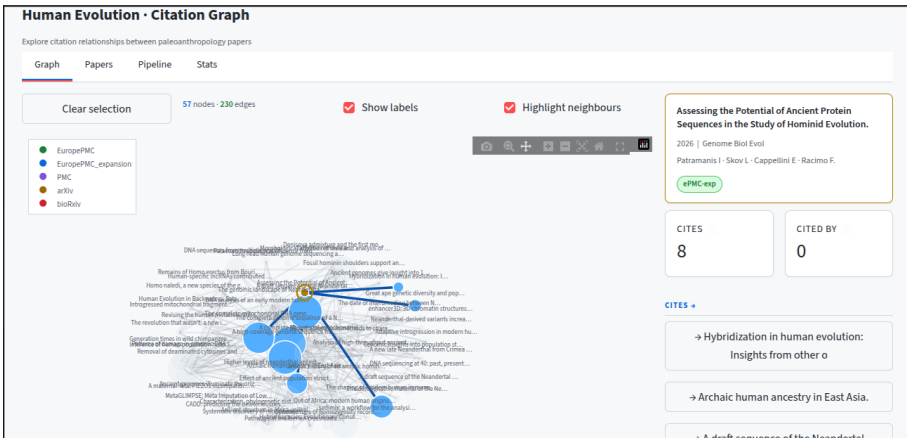

Figure 1: Streamlit dashboard showing the ego network of selected papers.

5.2 Use case – identifying hidden influential papers

By analysing betweenness centrality, we identified several papers that act as bridges between different subcommunities (e.g., between fossil morphology and population genetics). One such paper, although cited only 12 times, has a betweenness centrality in the top 5%, highlighting its role in connecting the two fields. This demonstrates how citation graph analysis can reveal non-obvious intellectual bridges.

6 Conclusion and Outlook

We have developed a fully functional citation graph builder tailored to human evolution research. The system successfully collects metadata, extracts references, resolves ambiguous citations, and provides an interactive dashboard for exploration. Future work includes:

- Adding full-text PDF parsing as a fallback for papers without API-accessible reference lists.
- Integrating temporal analysis (e.g., citation burst detection) and topic modelling of abstracts.

- Supporting incremental updates so that new papers can be added without rebuilding the entire graph.
- Deploying the dashboard as a public web service.

The code and data are freely available, and we hope the tool will assist researchers in navigating the growing literature on human evolution.

7 Acknowledgements

We thank the developers of Europe PMC, PubMed Central, arXiv, bioRxiv, and Semantic Scholar for providing open APIs. Thank you to Data Science Practice course for this innovative project idea.

8 Author Contributions

Anusa: Data collection, API integration, relevance filtering, analysis, README, testing, final report
Harshiddh: Graph construction, SQLite storage, NetworkX analysis.
Aritra: Dashboard design, visualisation, Streamlit implementation. **Aditi**: Report

9 Code availability

The complete source code is available at the following GitHub repository:
https://github.com/Anusa-Maiti/Team10_CitationGraphBuilder.git

10 References

Available in report/refs.bib

11 Appendix

11.1 Additional figures

Additional visualisations (degree distributions, source charts and temporal citation patterns) are available in the `figs/` folder of the repository.

11.2 Code snippets

The following Python snippets show how (1)metadate was fetched from arXiv (2)nodes and edges were constructed from the corpus. To compile this document with syntax highlighting, run `pdflatex -shell-escape report.tex`.

```python

def fetch_metadata(self, candidate: dict) -> dict | None:
        """arXiv search results are already metadata-rich - just normalise."""
        if not candidate:
            return None

        return {
            "arxiv_id":  candidate.get("arxiv_id"),
```

```
            "doi":       f"10.48550/arXiv.{candidate['arxiv_id']}" if candidate.get("arxiv_id") else
            "title":     candidate.get("title"),
            "authors":   candidate.get("authors", []),
            "year":      candidate.get("year"),
            "venue":     "arXiv (" + ", ".join(candidate.get("categories", [])) + ")",
            "abstract":  candidate.get("abstract"),
            "_pdf_url":  candidate.get("pdf_url"),  # internal - used by download_pdf
            "source":    "arXiv",
        }

def build_networkx_graph():
    """Build a NetworkX DiGraph from the CSVs."""
    meta = load_metadata_cached()
    refs = load_references_cached()

    G = nx.DiGraph()

    if meta.empty:
        return G

    # Add nodes
    for _, row in meta.iterrows():
        pid = row.get("paper_id", "")
        if not pid:
            continue
        G.add_node(pid, **{
            "title":    row.get("title",""),
            "year":     row.get("year",""),
            "venue":    row.get("venue",""),
            "authors":  row.get("authors",""),
            "doi":      row.get("doi",""),
            "pmid":     row.get("pmid",""),
            "source":   row.get("source",""),
            "abstract": row.get("abstract_snippet",""),
            "has_pdf":  row.get("has_pdf","no"),
            "citation_count": row.get("citation_count",""),
        })

    # Add edges - only resolved
    if not refs.empty:
        resolved = refs[refs["resolution_method"] != "unresolved"]
        seen = set()
        for _, row in resolved.iterrows():
            src = row.get("citing_paper_id","")
            tgt = row.get("cited_paper_id","")
            if not src or not tgt:
                continue
            if src not in G.nodes or tgt not in G.nodes:
                continue
            key = (src, tgt)
            if key in seen:
                continue
            seen.add(key)
            G.add_edge(src, tgt,
                       citing_title=row.get("citing_title",""),
                       cited_title=row.get("cited_title",""),
                       resolution_method=row.get("resolution_method",""))

```

```
    return G

```